

PBT-Bench: Benchmarking AI Agents on Property-Based Testing

Lucas Jing*

Tsinghua University
jgh23@mails.tsinghua.edu.cn

Xinqi Wang*

University of Washington
wxqkaxdd@cs.washington.edu

Liao Zhang

Beneficial AI Foundation
Liaoyuan Technology
zhangliao714@gmail.com

Simon S. Du

University of Washington
ssdu@cs.washington.edu

Abstract

Existing code benchmarks measure whether an agent can produce *any* test that reproduces a known bug, or whether it can produce a patch that fixes a described issue. Neither isolates the distinct skill of *property-based testing*: deriving a semantic invariant from documentation, and then constructing an input-generation strategy precise enough to make a random search reveal the violation. We introduce *PBT-Bench*, a benchmark of 100 curated property-based testing problems across 40 real Python libraries. Each problem injects one or more semantic bugs (365 in total, mean 3.65 per problem) designed so that default-strategy random inputs almost never trigger them; the agent must read the library’s documentation, identify the relevant invariant, and specify a Hypothesis `@given` strategy that concentrates mass in the trigger region. Bugs are stratified across three difficulty levels (L1–L3) spanning single-constraint boundary bugs to stateful, cross-function protocol violations. We evaluate eight contemporary LLMs under two prompting regimes (open-ended baseline vs. explicit Hypothesis scaffolding) for three independent runs per configuration. Bug recall under the PBT-guided prompt ranges from 42.1% to 83.4% across models; under the open-ended baseline, from 31.4% to 76.7%. Hypothesis scaffolding lifts mid-capability models by over 20 percentage points, but yields smaller gains for the strongest models, with two exceptions showing degradation, suggesting the structured prompt can interfere with certain model behaviours rather than complementing them. The hardest bugs prove model-specific: different architectures fail on different problems, leaving persistent gaps that no single model closes. We release the benchmark, harness, and full evaluation corpus to support downstream work on documentation-grounded semantic reasoning.

1 Introduction

Modern language models pass most code generation benchmarks when the expected behavior is stated concretely through input–output pairs. They do less well when the specification is *abstract*: a documented invariant that must hold across an entire input distribution rather than on any particular example. Property-based testing (PBT), introduced by QuickCheck [Claessen and Hughes, 2000] and popularized in Python by Hypothesis [MacIver et al., 2019], is a testing methodology in which a programmer specifies (i) a *property* — a predicate that should hold universally — and (ii) a *strategy*

*Equal contribution.

†Work done during an internship at the University of Washington.

```

1 from hypothesis import given, strategies as st
2
3 def qsort(xs):
4     if not xs: return []
5     p = xs[0] # pivot element at each recursion step
6     return qsort([x for x in xs[1:] if x < p]) + [p] + \
7            qsort([x for x in xs[1:] if x > p]) # BUG: ignores
            elements equal to pivot
8
9 @given(st.lists(st.integers()))
10 # Built-in strategy generating arbitrary integer lists
11 def test(xs):
12     ys = qsort(xs)
13     # properties: sorted output + multiset equivalence with input
14     assert is_sorted(ys) and same_multiset(xs, ys)

```

Figure 1: A Hypothesis property test exposing a quicksort bug: `@given(st.lists(st.integers()))` generates inputs with duplicates that trigger the missing-pivot-element defect, caught by the sortedness and multiset-equivalence properties.

— a typed, composable random-input generator — for sampling from the intended input distribution. The framework then searches for inputs that violate the property and shrinks any counterexample to a minimal failing form. A primer on Hypothesis strategies is in Appendix A.2. A minimal example of PBT is shown in Figure 1. We consider a buggy implementation of quicksort (`qsort`) that omits elements equal to the pivot at each recursive step. We verify two properties: the output is sorted, and it is a permutation (i.e., a multiset equivalent) of the input. We use the built-in `lists` strategy to sample random integer lists as inputs, which can generate cases with duplicate pivot values and thus trigger the assertion failure.

PBT provides greater coverage through random input exploration and expresses specifications as machine-checked, living documentation [Ravi and Coblenz, 2025]. However, writing a good property test is harder than a unit test [Ravi and Coblenz, 2025, Goldstein et al., 2024]: the author must abstract from concrete outputs to invariants and design an input distribution that targets likely violations—a barrier that LLMs are naturally positioned to lower.

Recent benchmarks have begun to probe LLMs’ testing abilities, but none isolates the PBT skill. SWE-BENCH [Jimenez et al., 2024] rewards *patch synthesis* given a bug report; SWT-BENCH [Mündler et al., 2024] rewards *test synthesis* given a bug report; TESTEXPLORA [Microsoft Research, 2026] rewards test synthesis from documentation, but over a mined distribution of real-world pull requests that is dominated by bugs whose fixes include dedicated unit tests, I/O coupling, or integration-level failures. Terminal-Bench [Merrill et al., 2026] evaluates agents end-to-end on terminal tasks with hand-crafted solutions but does not target any specific testing skill. *Crucially, even when existing benchmarks withhold explicit bug reports, they only require agents to write finite, concrete test cases. None evaluates the conceptual leap unique to PBT: translating an abstract specification into universal invariants and programmatic input distributions.*

We argue that evaluating PBT as a distinct skill requires curated — not mined — bugs, because not every bug is PBT-testable. A bug is PBT-testable only if (a) a semantic invariant of the API is violated, (b) the violation manifests under some expressible Hypothesis strategy without environmental mocks, and (c) the violation is deterministic under random input sampling. In practice, many real-world pull request bugs do not satisfy at least one of these conditions — common examples include UI rendering failures, database state dependencies, network ordering effects, and type errors that lack a corresponding documented invariant. A benchmark mined from PR history therefore conflates PBT skill with the agent’s ability to navigate around non-PBT-testable cases.

Contributions.

1. PBT-Bench, a suite of 100 property-based testing problems across 40 real Python libraries (serialization, data structures, date-time, numerics, type systems, state machines, parsing), each designed and verified to satisfy the PBT-testability conditions above. Every bug is accompanied

by a ground-truth Hypothesis strategy, a precise trigger condition, and a three-level difficulty label (L1–L3). To the best of our knowledge, this is the first benchmark that evaluates coding agents’ ability to use PBT for bug detection in program libraries.

2. An automated, containerized $F \rightarrow P$ (fail on buggy, pass on fixed) harness that evaluates each test function independently per injected bug, so that a test file covering multiple bugs is scored per-bug and not globally.
3. A large-scale evaluation: eight contemporary LLMs (Claude Sonnet 4.6, DeepSeek V3.2, Gemini 3 Flash, GLM 5.1, Grok 4.1 Fast, Qwen 3.6 Plus, Qwen 3.5-30B-A3B, Step 3.5 Flash), each run in two prompting regimes (open-ended baseline and explicit Hypothesis scaffolding) for three independent runs per cell, yielding 4,800 agent trajectories.
4. Three findings that appear specific to PBT evaluation: (i) the baseline-vs-PBT prompt gap is largest for models with weaker baselines (+24.5pp, +22.9pp, +20.3pp) and *smaller or inverted* for models with stronger baselines (+6.7pp for Sonnet 4.6; −3.2pp for DeepSeek V3.2; −8.0pp for Grok 4.1 Fast), consistent with scaffolding substituting for a capability weaker models lack, rather than complementing a skill stronger models already carry; (ii) on L2–L3 bugs no single model dominates — for a substantial fraction of problems the strongest overall model is matched or exceeded by weaker ones, suggesting PBT skill is not fully predicted by general coding ability; (iii) the reliable union recall across all sixteen model-mode pairs reaches 99.5% — 12.7 percentage points above the best single cell (86.8%, Sonnet 4.6 PBT) — and only two of 365 bugs are never reliably found by any cell (Appendix A.11), quantifying the ensemble headroom the benchmark exposes.

We release the benchmark, harness, problem templates, and full evaluation corpus under a permissive licence. All problem files contain the BIG-BENCH canary string to aid decontamination of future training corpora [Srivastava et al., 2023].

2 Related work

Benchmarks for patch synthesis. SWE-BENCH [Jimenez et al., 2024] established the template for modern coding-agent evaluation: given a GitHub issue and the corresponding repository snapshot, an agent must produce a patch that passes a held-out reference test. SWE-BENCH VERIFIED [OpenAI and Princeton NLP, 2024] curates a higher-signal subset, and more recent variants (SWE-BENCH MULTIMODAL [Yang et al., 2025], SWE-LANCER [Miserendino et al., 2025], SWE-BENCH PRO/EVO [Thai et al., 2025]) extend scope and difficulty. Verified has effectively saturated (top scores exceed 85% by early 2026), which has motivated both harder patch benchmarks and orthogonal skill benchmarks like ours. PBT-Bench does *not* ask the agent to fix the bug; it asks only whether the agent can *expose* it via a property test. Repair and exposure are known-distinct skills [Mündler et al., 2024].

Benchmarks for test synthesis. SWT-BENCH [Mündler et al., 2024] re-frames SWE-bench for test generation: given the issue description and buggy code, generate a test that fails on the buggy version and passes after the published fix. The fail-to-pass ($F \rightarrow P$) criterion introduced there is the foundation of our own per-function evaluation; it traces to mined bug corpora such as DEFECTS4J [Just et al., 2014] and BUGSINPY [Widyasari et al., 2020], though mining from version history does not guarantee PBT-testability. Two design differences distinguish PBT-Bench: (i) we withhold the bug description from the agent — the only oracle is the library’s own documentation — and (ii) we require the test to be a Hypothesis `@given` property, not an arbitrary concrete assertion. This precludes a class of trivial passes where the agent encodes the fixed behavior’s observable output as a literal.

Documentation-as-oracle test generation. The closest benchmark in framing is TESTEXPLORA [Microsoft Research, 2026], which mines 2,389 doc-as-oracle test-generation tasks from GitHub pull requests. Its bug distribution is natural rather than PBT-filtered, so many tasks are concrete-assertion-testable or I/O-coupled; PBT-Bench complements it by isolating the PBT-specific skill on a controlled distribution.

General-purpose agent benchmarks. TERMINAL-BENCH 2.0 [Merrill et al., 2026] evaluates agents on 89 hard terminal tasks; its construction protocol—three-reviewer verification, adver-

serial audits, and oracle-solution integrity—directly informs our QA pipeline (Section 3). HUMAN-EVAL [Chen et al., 2021], LIVECODEBENCH [Jain et al., 2025], CLASSEVAL [Du et al., 2024], and AUTOCODEBENCH [Chou et al., 2025] target code synthesis, not testing or bug exposure.

Property-based testing and LLMs. LLM-based concrete test generators (CHATUNITEST [Chen et al., 2024], COVERUP [Pizzorno and Berger, 2024], CODAMOSA [Lemieux et al., 2023], LIBRO [Kang et al., 2023]) produce input–output pairs; PBT-Bench requires instead a universal invariant paired with a typed Hypothesis strategy—a qualitatively different output. PBT originated with QuickCheck in Haskell [Claessen and Hughes, 2000], enabling automated random test execution against formal specifications. It has since been ported to many languages, including Python (Hypothesis [MacIver et al., 2019]) and Scala (ScalaCheck [Sood, 2016]), and adopted in industrial settings for increasing testing confidence and finding edge-case bugs [Ravi and Coblenz, 2025]. Researchers have recently explored various approaches to integrate PBT with LLMs. He et al. [2025] demonstrates that validating LLM outputs against PBT invariants improves pass@1 accuracy for code generation. Vikram et al. [2023] studies whether LLMs can generate property-based tests from API documentation, evaluating test quality against LLM-generated code mutants. While these works explore LLM-generated PBT, their evaluation relies on artificial mutants or simplified test targets, leaving it unclear whether agents can discover actual, stealthy semantic bugs in complex libraries. PBT-Bench bridges this gap by providing a rigorous $F \rightarrow P$ benchmark with 365 human-verified, deeply-injected bugs that closely mimic real-world invariant violations, evaluated on autonomous coding agents rather than static API calls. Xiong et al. [2026] improves PBT property generation via a multi-stage LLM workflow on Android applications [Xiong et al., 2024]. Unlike PBT-Bench, it uses static LLM API calls rather than coding agents and does not compare against direct LLM bug detection. LLM-augmented fuzzers such as TITANFUZZ [Deng et al., 2023] and FUZZGPT [Deng et al., 2024] similarly exploit LLMs to generate inputs that expose bugs in deep-learning libraries, but operate at the level of raw programs without typed strategies or falsifiable predicates—the distinctions that make PBT a *documented*, reusable specification rather than a one-shot witness. A concurrent report [Maaz et al., 2025] deploys an LLM agent in an open-ended PBT loop to find bugs in real Python packages; unlike that tool-oriented study, PBT-Bench uses injected, verified bugs with a fixed $F \rightarrow P$ harness, enabling controlled cross-model comparison.

3 The PBT-Bench Benchmark

PBT-Bench contains 100 curated property-based testing problems across 40 Python libraries. This section specifies the problem format (§3.1), the three-level difficulty taxonomy (§3.2), the curation and quality-assurance protocol (§3.3), and the fidelity argument for injected bugs (§3.4).

3.1 Problem format

Each problem contains a buggy library, agent-accessible documentation (`docs/`; the sole oracle), a passing existing test suite, and one or more reversible bug patches (`bug_N.patch`). Reversibility is key: the default container state applies all patches; removing patch N yields the fixed-bug- N configuration, enabling independent per-bug $F \rightarrow P$ scoring even when a problem contains multiple co-injected bugs. Full directory layout and metadata schema are in Appendix A.1.

PBT-testability criteria. Every injected bug must satisfy four criteria, which are enforced during curation and re-verified by `check_infra.py`. Specifically, we require *semantic*, meaning the bug violates a documented invariant or contract rather than an implementation detail, including properties such as roundtrip identity, commutativity, monotonicity, bounded output ranges, and protocol preconditions; *expressivity*, meaning the invariant can be encoded as a `@given` property using Hypothesis’s built-in and `@composite` strategies without relying on external effects such as files, sockets, or clocks; *stealth*, meaning the bug is not easily detectable by short manual inspection and typically resides in under-tested code paths or is masked by locally plausible computations; and *deterministic trigger*, meaning there exists a well-defined input region T on which the bug manifests with probability one (or a specific action sequence in the stateful case), with the ground-truth strategy concentrating probability mass on T .

Two-container evaluation harness. The benchmark is run in two independent containers per problem (Docker or Apptainer, depending on the host environment). The first (*agent container*) contains the buggy library, documentation, and existing test suite, and is handed to the agent under the OpenHands SDK [Wang et al., 2025] with the prompt template of the relevant mode. The second (*eval container*) is created after the agent writes `pbt_test.py`: for each injected bug b , the harness restores the buggy library, runs each test function, then reverses only patch b and re-runs. A bug is *found* when at least one test function passes the $F \rightarrow P$ predicate — fails on the buggy version and passes on the version with b reversed — for that specific bug. This per-function, per-bug scoring correctly credits agents who write one test per bug in a single file.

3.2 Difficulty taxonomy

In Hypothesis, a *strategy* (`SearchStrategy`) draws random values from a typed search space and automatically shrinks any discovered counterexample to a minimal failing form. The `@given(strat, ...)` decorator binds strategies to a test function’s arguments and drives the adaptive search engine; built-in primitives (`st.lists()`, `st.integers()`, `st.floats()`, etc.) and the `@st.composite` decorator cover most input structures. A fuller primer is in Appendix A.2. Each injected bug is labelled with one of three difficulty levels reflecting the minimum Hypothesis-strategy sophistication required for random search to hit the trigger region in under 200 examples. **L1** corresponds to single-constraint bugs, where either Hypothesis’s default strategies suffice (e.g., boundary exploration triggers the bug with high probability) or the agent needs to identify a single input constraint such as a specific value range, size class, or structural property; the trigger probability under default strategies ranges from high to below one percent. **L2** denotes multi-constraint triggers that require simultaneously satisfying several documentation-derived conditions, such as value ranges, structural constraints, and operation ordering. **L3** captures cross-function or cross-operation protocol violations, which require tests over sequences of API calls or metamorphic tests (i.e., checking invariants across multiple executions), where the invariant spans multiple API calls.

Across the 100 canonical evaluation problems, the 365 injected bugs distribute as follows: L1 = 87 (24%), L2 = 184 (50%), L3 = 94 (26%). The distribution is intentionally skewed toward L2–L3: early curation rounds consistently found that trivial single-constraint bugs were solved by both reference models in a single rollout, providing no discriminative signal; most were retired or relabelled.

3.3 Curation and quality assurance

Library selection. Libraries were selected for rich internal-state invariants not directly observable through the public API, a substantial gap between the public interface and its implementation (making code-reading localization hard), documentation quality sufficient for doc-first trigger inference, and pure Python implementation. We excluded encode/decode core paths (roundtrip properties trivially cover their bugs) and libraries where patches are likely to conflict with internal self-use.

Bug design and retirement. Problems were developed through an iterative design-evaluate-retire process in which an LLM-assisted design agent proposed candidate bugs and patches, and human authors reviewed each candidate for correctness, stealth, and PBT-testability before acceptance. After each design round, every candidate problem was evaluated against two *reference models* (Claude Sonnet 4.6 and GLM 5.1, each running the OpenHands baseline scaffold for a single rollout); problems on which both reference models achieved full recall were retired or redesigned, as unanimous success on a single rollout provides no discriminative signal. The difficulty bar was raised progressively across rounds as earlier problems proved easier than anticipated. Retired problems are archived in the repository as documented negative examples for future benchmark construction.

Each bug candidate was checked against a design checklist before acceptance: injection at least two call-chain layers below the public API surface; domain knowledge required to understand why the code is incorrect (the bug must not be obviously wrong from a local reading); the *clean-pass criterion* (the injected bug must leave the library’s existing unit test suite intact); for problems containing multiple bugs, each bug must be independently triggerable by a distinct input region that does not simultaneously trigger other bugs in the same problem (verified via pairwise testing of ground-truth strategies); and the existence of a ground-truth strategy derivable from documentation alone that finds the bug within the default example budget.

Three-stage QA pipeline. Each accepted problem passed three verification stages. *Stage 1 (automated)* ran `check_infra.py` to verify clean patch application, upstream-test pass, trigger-path independence, and ground-truth solution validity. *Stage 2 (reference-model gate)* evaluated each problem under the two reference models defined above; problems where both achieved full recall were retired. *Stage 3 (manual adversarial check)* confirmed that each accepted bug satisfies the stealth criterion and that existing-test pass rates are unaffected. The Stage 3 protocol is the same one applied in Section 6.

3.4 Fidelity to real-world bugs

Controlled bug injection raises a representativeness concern. Our position: *PBT-testability is a design constraint, not a distributional one* — a benchmark evaluating PBT must select for bugs PBT could in principle detect. Within that constraint we preserve fidelity three ways. First, every injected bug falls into one of eleven classes empirically observed in open-source Python bug reports (off-by-one/boundary, wrong reference/expression, condition/logic inversion, wrong operator, argument/field order swap, missing operation, traversal/ordering, sign/direction error, protocol/domain-specific, wrong constant/value, and state corruption/mutation); the full taxonomy with counts is in Appendix A.5. Second, every bug satisfies the *clean-pass criterion* (defined in Section 3.3), matching the empirical fact that released semantic bugs escaped their own tests. Third, the 40 libraries span seven domains (serialization, data structures, date-time, type systems and schemas, numerics, state machines, parsing/HTML); the full library list is in Appendix A.4.

4 Experimental setup

We evaluate eight contemporary LLMs under two prompting regimes, each repeated three times per problem, over the 100-problem evaluation set. All configurations use the same OpenHands agent scaffold to isolate prompt and model effects from scaffold-specific tuning.

Models. We evaluate Claude Sonnet 4.6, DeepSeek V3.2, Gemini 3 Flash, GLM 5.1, Grok 4.1 Fast, Qwen 3.6 Plus, Qwen 3.5-30B-A3B (Mixture-of-Experts, 3B active parameters), and Step 3.5 Flash. All models are queried through OpenRouter with the provider’s default temperature. Model release dates span April 2025 through February 2026.

Prompt regimes. Each model is evaluated under two prompts:

- **Baseline.** An open-ended four-phase instruction — understand the library, find uncovered behavior, write a failing test, finalize — that does not mention property-based testing, Hypothesis, or `@given`. The agent is told only that a test must fail on the current library and pass after the bug is fixed. This measures general testing ability.
- **PBT.** An explicitly structured four-step instruction that names Hypothesis as the testing framework, requires `@given` decorators, and teaches the Roundtrip/Model-oracle/Invariant property taxonomy. The agent is instructed to write at least five focused test functions, each covering one documented invariant, without first hunting for a specific bug. This measures PBT ability under scaffolding.

The two prompt templates are reproduced verbatim in Appendix A.3. The comparison is an ablation of prompt-level scaffolding with all other variables held fixed. We verified post-hoc that no Baseline trajectory independently installed or imported Hypothesis, confirming the two conditions constitute a clean prompt ablation.

Agent scaffold. All models run inside the OpenHands SDK v1.11.5 with the standard tool set (file editor, terminal, think) and a maximum of 200 tool-call iterations. Agent containers are built from `python:3.12-slim` with the target library preinstalled via a cached image layer per library. PBT-mode containers additionally preinstall Hypothesis; Baseline containers do not, though the agent may install it during its run. Each agent has 60 minutes of wall-clock time per problem.

Evaluation. After the agent writes `/workspace/pbt_test.py`, the harness collects the set of test functions, then for each injected bug b_i : (i) restores the buggy library from a pristine backup, (ii) runs each test function individually with a per-function timeout (120 s for Baseline, 300 s for

Table 1: Headline results per model and prompt regime, averaged over three runs on all 100 problems. **Recall** is bug-level; Δ is PBT–Baseline recall in percentage points (shown once per model, negative values indicate PBT hurts); $F \geq 1$ is the rate of problems with at least one bug found; **Perfect** is the rate of problems with all bugs found. Recall, $F \geq 1$, and Perfect values are point estimates followed by the half-width of percentile 95% bootstrap confidence intervals (CIs), based on 1,000 resamples, in pp. Best value in each column is bold.

Model	Mode	Recall	Δ	$F \geq 1$	Perfect	Turns
Sonnet 4.6	PBT	83.4% ± 3.3	+6.7	92.7% ± 2.8	67.0% ± 5.2	68.4
Sonnet 4.6	Baseline	76.7% ± 2.7		98.0% ± 1.5	43.0% ± 5.8	42.5
Qwen 3.6 Plus	PBT	78.0% ± 3.5	+24.5	90.3% ± 3.2	56.0% ± 5.7	45.1
Qwen 3.6 Plus	Baseline	53.5% ± 3.8		85.3% ± 4.0	18.3% ± 4.2	27.1
GLM 5.1	PBT	72.1% ± 4.5	+5.5	77.0% ± 4.8	63.0% ± 5.0	89.2
GLM 5.1	Baseline	66.6% ± 4.3		79.7% ± 4.5	44.0% ± 5.3	57.4
DeepSeek V3.2	PBT	61.0% ± 4.4	−3.2	79.7% ± 4.5	38.7% ± 5.7	50.9
DeepSeek V3.2	Baseline	64.2% ± 3.8		90.0% ± 3.5	35.3% ± 5.5	61.8
Gemini 3 Flash	PBT	62.8% ± 3.8	+6.6	84.0% ± 4.0	37.0% ± 5.7	34.0
Gemini 3 Flash	Baseline	56.2% ± 4.7		71.3% ± 5.2	38.3% ± 5.7	44.6
Step 3.5 Flash	PBT	58.1% ± 4.9	+20.3	70.7% ± 5.2	41.0% ± 5.5	83.4
Step 3.5 Flash	Baseline	37.8% ± 3.7		67.3% ± 5.3	14.0% ± 4.0	51.0
Qwen 3.5-30B-A3B	PBT	54.3% ± 4.4	+22.9	77.3% ± 4.8	25.7% ± 4.8	48.1
Qwen 3.5-30B-A3B	Baseline	31.4% ± 2.9		76.0% ± 4.7	5.3% ± 2.7	30.2
Grok 4.1 Fast	PBT	42.1% ± 4.3	−8.0	63.0% ± 5.5	21.0% ± 4.7	31.8
Grok 4.1 Fast	Baseline	50.1% ± 3.8		88.3% ± 3.5	20.7% ± 4.7	33.3

PBT), (iii) reverses patch b_i , (iv) re-runs each test function. A function is *useful* for b_i if it fails on the buggy version and passes after b_i is reversed. The bug is *found* if at least one function is useful for it. Our primary metrics are:

- (1) **Bug recall** — bugs_found/bugs_total, averaged across problems; the headline metric.
- (2) **Problem coverage** — fraction of problems on which at least one bug is found; measures breadth.
- (3) **Full recall rate** — fraction of problems on which *all* bugs are found; measures depth.
- (4) **Test precision** — useful_test_functions/total_test_functions; a per-function quality signal independent of recall.

Reproducibility and decontamination. All 4,800 trajectories, generated pbt_test.py files, and container build scripts are released under a permissive licence; one trajectory timed out and is recorded as not-found (Appendix A.14). Every problem file embeds the BIG-BENCH canary GUID [Srivastava et al., 2023] for training-corpus decontamination. Computational details and API costs are in Appendix A.15.

5 Results

We report results from 4,800 agent trajectories: eight models, two prompt regimes, three independent runs per cell, 100 problems per run. Unless otherwise stated, numbers in this section are averaged across the three runs.

5.1 Headline results

Table 1 reports the four primary metrics per (model, mode) cell. Three observations stand out.

(1) PBT scaffolding helps mid-capability models most. The recall gap Δ = PBT – Baseline ranges from +24.5pp (Qwen 3.6 Plus) down to −3.2pp (DeepSeek V3.2) and −8.0pp (Grok 4.1 Fast). Sonnet 4.6 sits at +6.7pp: PBT mode helps, but by less than the gap for Qwen 3.5-30B-A3B (+22.9pp). For the four models with the largest gaps (Qwen 3.6, Qwen 3.5-30B-A3B, Step, Sonnet), $|\Delta|$ exceeds the sum of the two cells’ 95% bootstrap CIs, so those signs are robust to resampling; for the remaining four models, the gaps are smaller than the CI widths and should be

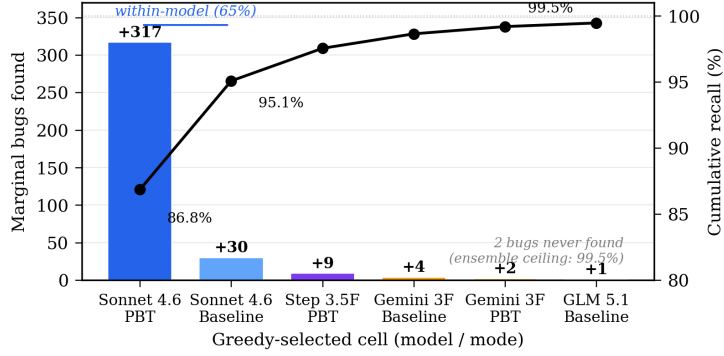


Figure 2: Greedy ensemble construction: marginal bugs found per cell (bars, left axis) and cumulative recall (line, right axis). Only 6 of 16 cells contribute unique bugs; within-model mode complementarity (Sonnet PBT + Baseline) accounts for 65% of the marginal gains.

interpreted directionally. This suggests the prompt’s explicit property taxonomy and @given template is substituting for capability a weaker model lacks — not complementing a skill a stronger model already has. We analyze this further in Section 6.

(2) Problem coverage and full recall diverge; the benchmark is not saturated. Sonnet 4.6 Baseline hits the highest *problem coverage* (98.0%) but a *full recall* rate of only 43.0%, frequently missing co-located bugs; PBT mode trades a small coverage drop (92.7%) for a large gain in full recall (67.0%), consistent with its instruction to write multiple focused tests rather than hunt for a single bug. No cell achieves full recall above 70%: Sonnet 4.6 PBT leaves 33.0% of problems partially unsolved, and for every model-mode pair at least one problem per run yielded no F→P test.

5.2 Ensemble upper bound

To separate capability limits from problem diversity, we define *reliable coverage*: a bug is reliably covered by a cell if found in at least two of three independent runs, smoothing over Hypothesis’s search stochasticity.

Figure 2 shows the greedy union curve. The best single cell is Claude Sonnet 4.6 PBT, which reliably covers 317 of 365 bugs (86.8%); Qwen 3.6 Plus PBT follows at 303 (83.0%). Greedily adding a second cell (Claude Sonnet 4.6 Baseline) lifts union coverage to 95.1%; the full 16-cell ensemble reaches 99.5%. Two bugs — BIDD-004 bug_3 (a stateful bidict invariant violation) and TMLK-005 bug_1 (a tomlkit container indexing bug) — are never reliably found by any cell; both are L3 bugs whose trigger requires a precise multi-step state sequence that no model reliably constructs within the 200-example budget (details in Appendix A.11). Two observations stand out. First, within-model mode complementarity provides the majority of the ensemble headroom: the same model in two modes (Sonnet PBT + Sonnet Baseline) accounts for 30 of the 46 marginal bugs (65%), lifting coverage from 86.8% to 95.1%. Cross-architecture additions contribute modestly (Step +9, Gemini +6, GLM +1), reaching the 99.5% ceiling. This suggests PBT and Baseline elicit complementary testing strategies even within a single architecture. Second, at the bug level, 2/365 bugs remain out of reach of any cell within our max-iterations budget; at the problem level, the hardest-problem distribution (Appendix A.10) is richer.

6 Analysis

Baseline vs. PBT: scaffolding as capability supplement. Section 5.1 noted that PBT scaffolding benefits mid-tier models far more than frontier ones, with the strongest model (Sonnet 4.6) seeing a +6.7pp bump and the weakest mainstream model (Qwen 3.5-30B-A3B, 3B active parameters) seeing +22.9pp. A plausible mechanism is that the PBT prompt provides three things the baseline does not: (i) an explicit framework choice (Hypothesis) that eliminates a planning step, (ii) a worked code template (@given(...) ... @settings(max_examples=200, deadline=None)),

and (iii) a compact property taxonomy (Roundtrip, Model-oracle, Invariant, Boundary) that scaffolds the “what should I test” decision.

Consistent with this, PBT mode raises Qwen 3.5-30B-A3B’s test-file completion rate from 84% to 91% and average tool-call count from 29 to 48.

Diagnostics: difficulty, complementarity, failure modes. Three short diagnostics support the headline numbers (full tables in Appendices A.9–A.12). (1) *L1–L3 labels track empirical hardness*: Sonnet-Baseline recall descends monotonically with L1/L2/L3 (79%/78%/68%; $n = 87/184/94$); this ordering holds in 15 of 16 cells (Table 3), and averaged across all eight models the mean recall is 66%/60%/51%. The L1/L2 gap is narrower than the L2/L3 gap as expected: both L1 and L2 require input-constraint reasoning (differing only in constraint count), while L3 additionally requires cross-function protocol understanding. (2) *No model dominates the hardest bugs*: the three GALS finite-field problems are each solved at different rates by different model families, so a cross-model union is required to cover them. (3) *Baseline and PBT fail categorically differently*. Over 20 sampled failures per (model, mode) cell classified into eight mutually-exclusive modes, Baseline failures are dominated by **Incorrect Assertion** (59%) and **Overly Concrete Test** (32%) — correct-looking tests aimed at the wrong invariant or missing the trigger region. PBT failures instead shift to search pathologies: **Assume Misuse** (31% overall, up to 75% for GLM 5.1), where the agent calls `hypothesis.assume()` on a predicate that excludes the bug’s trigger region, and **Wrong Strategy Range** (31%, up to 50% for Gemini 3 Flash). The **Assume Misuse** diagnostic is prompt-actionable: an explicit warning against `assume()`-based filtering should recover a substantial fraction of PBT-mode failures. (4) *Scaffolding benefit is not uniform across difficulty levels* (Table 3). For weaker models the Δ decreases with difficulty: Qwen 3.5-30B-A3B gains +35.2/+25.4/+14.8 pp on L1/L2/L3, suggesting the prompt substitutes for constraint-reasoning on easy bugs but cannot close the gap on L3 protocol violations. Step 3.5 Flash is the exception, showing a near-flat benefit ($\approx +22.5$ pp) across all three levels. Models for which PBT hurts overall show degradation concentrated on harder bugs: DeepSeek V3.2 shows +2.7/−4.0/−7.8 pp, and Grok 4.1 Fast −5.7/−5.2/−10.6 pp, a pattern consistent with the scaffold constraining rather than guiding their search on L2–L3 problems. Full per-cell breakdowns and analysis are in Appendix A.9.

Adversarial audit and data-integrity disclosure. We ran two complementary integrity checks. (A) Under an explicit cheat prompt (adapted from Terminal-Bench’s exploited-audit protocol [Merrill et al., 2026]), Claude Sonnet 4.6 was re-run on 30 trajectories; recall averaged 0.85–0.93, matching normal Baseline performance, and inspection found *zero* attack signatures (no hardcoded assertions, version-gated conditionals, or path manipulation), confirming F→P robustness to prompt-level gaming. (B) During early runs the harness left `*.orig` backup files readable by agents; all affected trajectories (8.0%) were re-run on a fixed harness, and a paired test shows no significant recall effect (Wilcoxon $p = 0.31$); details are in Appendix A.7.

Run-to-run stability. Three independent runs per cell suffice for reliable comparison: the median 95% bootstrap CI half-width across all 16 cells is 3.8 pp (range 2.7–4.9 pp), and residual variance is attributable to Hypothesis’s internal search stochasticity rather than agent non-determinism. Full stability analysis and a post-hoc paraphrased-prompt sensitivity check are in Appendix A.13.

7 Conclusion

PBT-Bench probes a capability existing code-agent benchmarks implicitly assume: moving from a documented invariant to a Hypothesis strategy precise enough to expose a semantic bug. On 100 curated problems across 40 real Python libraries — all passing a PBT-testability screen and three-stage QA pipeline — property-based testing remains a persistent capability gap: the strongest current model reaches 83% bug-level recall, and the 16-cell union still fails to reliably find two of 365 bugs. Scaffolding toward Hypothesis disproportionately helps weaker-baseline models, indicating the prompt substitutes for a capability stronger models already carry in pretraining, rather than adding a skill. We release the benchmark, harness, and evaluation corpus; PBT-Bench complements test-synthesis benchmarks mined from PRs (TESTEXPLORA, SWT-BENCH) by measuring depth on exactly the bug distribution for which PBT is the intended tool. *Limitations.* Curated bugs trade population fidelity for PBT-testability (§3.4); PBT-Bench covers only Python libraries with English-language documentation, and generalization to other languages is untested. All agents run on a single

scaffold (OpenHands v1.11.5 with a fixed tool set); performance may differ under richer end-to-end systems such as Claude Code or Codex CLI that couple the LLM to a broader environment, and prompt sensitivity is only partially characterized (§A.13). The `max_examples=200` budget likely caps L2/L3 recall on narrow-trigger problems (e.g. GALS-003, where the bug manifests only for specific random seeds within the budget window); a controlled budget sweep is left to future work. *Future work.* Raising `max_examples` to 1,000 would disentangle search-budget from strategy-quality gaps on L2/L3 bugs; adding an explicit `assume()`-misuse warning to the PBT prompt is a low-cost intervention expected to recover a substantial fraction of PBT-mode failures.

Acknowledgments and Disclosure of Funding

SSD acknowledges the support of NSF IIS 2229881, NSF CIF 2212261, NSF IIS 2143493, UW-Tsukuba Amazon NVIDIA Cross Pacific AI Initiative (XPAI), and the AI2050 program at Schmidt Sciences.

References

- Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, et al. Evaluating large language models trained on code, 2021. URL <https://arxiv.org/abs/2107.03374>.
- Yinghao Chen, Zehao Hu, Chen Zhi, Junxiao Han, Shuiguang Deng, and Jianwei Yin. ChatUniTest: A framework for LLM-based test generation. In *Proceedings of the ACM International Conference on the Foundations of Software Engineering (FSE), Demonstrations*, 2024. doi: 10.1145/3663529.3663801.
- Jason Chou, Ao Liu, Yuchi Deng, et al. AutoCodeBench: Large language models are automatic code benchmark generators, 2025. URL <https://arxiv.org/abs/2508.09101>.
- Koen Claessen and John Hughes. Quickcheck: A lightweight tool for random testing of Haskell programs. In *Proceedings of the Fifth ACM SIGPLAN International Conference on Functional Programming (ICFP '00)*, pages 268–279, 2000.
- Yinlin Deng, Chunqiu Steven Xia, Haicheng Peng, Chenyuan Yang, and Lingming Zhang. Large language models are zero-shot fuzzers: Fuzzing deep-learning libraries via large language models. In *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*, 2023. doi: 10.1145/3597926.3598067.
- Yinlin Deng, Chunqiu Steven Xia, Chenyuan Yang, Shizhuo Dylan Zhang, Shujing Yang, and Lingming Zhang. Large language models are edge-case fuzzers: Testing deep learning libraries via FuzzGPT. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering (ICSE)*, 2024. doi: 10.1145/3597503.3623343.
- Xueying Du et al. ClassEval: A manually-crafted benchmark for evaluating llms on class-level code generation. In *First Conference on Language Modeling (COLM)*, 2024.
- Harrison Goldstein, Joseph W Cutler, Daniel Dickstein, Benjamin C Pierce, and Andrew Head. Property-based testing in practice. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, pages 1–13, 2024.
- Lehan He et al. Property-generated solver: Leveraging property-based testing to validate LLM code, 2025. URL <https://arxiv.org/abs/2506.18315>.
- Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. LiveCodeBench: Holistic and contamination free evaluation of large language models for code. In *The Thirteenth International Conference on Learning Representations (ICLR)*, 2025. URL <https://openreview.net/forum?id=chfJJYC3iL>.
- Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R. Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *The Twelfth International Conference on Learning Representations (ICLR)*, 2024. URL <https://openreview.net/forum?id=VTF8yNQm66>.

- René Just, Darioush Jalali, and Michael D. Ernst. Defects4J: A database of existing faults to enable controlled testing studies for Java programs. In *Proceedings of the International Symposium on Software Testing and Analysis (ISSTA)*, 2014. doi: 10.1145/2610384.2628055.
- Sungmin Kang, Juyeon Yoon, and Shin Yoo. Large language models are few-shot testers: Exploring LLM-based general bug reproduction. In *Proceedings of the 45th IEEE/ACM International Conference on Software Engineering (ICSE)*, 2023. doi: 10.1109/ICSE48619.2023.00194.
- Caroline Lemieux, Jeevana Priya Inala, Shuvendu K. Lahiri, and Siddhartha Sen. CodaMosa: Escaping coverage plateaus in test generation with pre-trained large language models. In *Proceedings of the 45th IEEE/ACM International Conference on Software Engineering (ICSE)*, 2023. doi: 10.1109/ICSE48619.2023.00085.
- Muhammad Maaz, Liam DeVoe, Zac Hatfield-Dodds, and Nicholas Carlini. Agentic property-based testing: Finding bugs across the Python ecosystem, 2025. URL <https://arxiv.org/abs/2510.09907>.
- David R. MacIver, Zac Hatfield-Dodds, et al. Hypothesis: A new approach to property-based testing. *Journal of Open Source Software*, 4(43):1891, 2019. doi: 10.21105/joss.01891.
- Mike A. Merrill, Alexander G. Shaw, Nicholas Carlini, et al. Terminal-Bench: Benchmarking agents on hard, realistic tasks in command line interfaces, 2026. URL <https://arxiv.org/abs/2601.11868>.
- Microsoft Research. TestExplora: Benchmarking LLMs for proactive bug discovery via repository-level test generation, 2026. URL <https://github.com/microsoft/TestExplora>. arXiv preprint, February 2026.
- Samuel Miserendino, Michele Wang, Tejal Patwardhan, and Johannes Heidecke. SWE-Lancer: Can frontier llms earn \$1 million from real-world freelance software engineering? In *Forty-second International Conference on Machine Learning (ICML)*, 2025. URL <https://openreview.net/forum?id=xZXhFg43EI>.
- Niels Mündler, Mark Niklas Müller, Jingxuan He, and Martin Vechev. SWT-Bench: Testing and validating real-world bug-fixes with code agents. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems (NeurIPS)*, 2024. URL <https://openreview.net/forum?id=9Y8zU011EQ>.
- OpenAI and Princeton NLP. SWE-bench Verified: a human-validated subset of swe-bench. <https://openai.com/index/introducing-swe-bench-verified/>, 2024. OpenAI blog post, 13 August 2024.
- Melissa Z. Pan, Mert Cemri, Lakshya A. Agrawal, et al. Why do multiagent systems fail? In *ICLR 2025 Workshop on Building Trust in Language Models and Applications*, 2025. URL <https://openreview.net/forum?id=wM521FqPvI>.
- Juan Altmayer Pizzorno and Emery D. Berger. CoverUp: Coverage-guided, LLM-based test generation, 2024. URL <https://arxiv.org/abs/2403.16218>.
- Savitha Ravi and Michael Coblenz. An empirical evaluation of property-based testing in python. *Proceedings of the ACM on Programming Languages*, 9(OOPSLA2):3897–3923, 2025.
- Gaurav Sood. *Scala Test-Driven Development*. Packt Publishing Ltd, 2016.
- Aarohi Srivastava et al. Beyond the imitation game: Quantifying and extrapolating the capabilities of language models. *Transactions on Machine Learning Research*, 2023. ISSN 2835-8856. URL <https://openreview.net/forum?id=uyTL5Bvosj>.
- Minh V. T. Thai, Tue Le, Dung Nguyen Manh, Huy Phan Nhat, and Nghi D. Q. Bui. SWE-EVO: Benchmarking coding agents in long-horizon software evolution scenarios. *arXiv preprint arXiv:2512.18470*, 2025. URL <https://arxiv.org/abs/2512.18470>.
- Vasudev Vikram et al. Can large language models write good property-based tests?, 2023. URL <https://arxiv.org/abs/2307.04346>.

- Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, et al. OpenHands: An open platform for AI software developers as generalist agents. In *The Thirteenth International Conference on Learning Representations (ICLR)*, 2025. URL <https://openreview.net/forum?id=0Jd3ayDDoF>.
- Ratnadira Widyasari, Sheng Qin Sim, Camellia Lok, Haodi Qi, Jack Phan, Qijin Tay, Constance Tan, Fiona Wee, Jodie Jetlee Tan, Yong Ding Yap, et al. BugsInPy: A database of existing bugs in Python programs to enable controlled testing and debugging studies. In *Proceedings of the 28th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*, 2020. doi: 10.1145/3368089.3417943.
- Yiheng Xiong, Ting Su, Jue Wang, Jingling Sun, Geguang Pu, and Zhendong Su. General and practical property-based testing for android apps. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*, pages 53–64, 2024.
- Yiheng Xiong, Ting Su, Jingling Sun, Jue Wang, Qin Li, Geguang Pu, and Zhendong Su. From natural language to executable properties for property-based testing of mobile apps. *arXiv preprint arXiv:2603.21263*, 2026.
- John Yang, Carlos E. Jimenez, et al. SWE-bench Multimodal: Do ai systems generalize to visual software domains? In *The Thirteenth International Conference on Learning Representations (ICLR)*, 2025.

A Additional experimental details

A.1 Problem format

Each problem is stored in `libraries/<lib>/problems/<ID>/` and consists of five components. `problem.yaml` defines metadata including library version, injected bugs with difficulty levels (L1–L3), trigger conditions, ground-truth Hypothesis strategies, and semantic tags (`prop:*`, `bug:*`, `strat:*`). Each injected bug is represented as a `bug_N.patch` unified diff applied at container startup, enabling reversible injection: the default state includes all patches; removing patch N yields the fixed-bug- N configuration for per-bug scoring. `docs/` provides agent-accessible documentation (official sources when available), serving as the only external oracle. `existing_tests/` contains unit tests that pass on the buggy version, ensuring injected bugs escape the baseline suite. `ground_truth/` includes reference solutions and strategy specifications used exclusively for offline validation and never exposed to the agent.

A.2 Hypothesis strategy primer

In Hypothesis, a *strategy* (`SearchStrategy`) is an object that draws random values from a defined search space and automatically shrinks any discovered counterexample to a minimal failing form. The `hypothesis.strategies` module (aliased `st` by convention) provides built-in primitives — `st.lists()`, `st.integers()`, `st.text()`, `st.floats()` — and the `@st.composite` decorator for constructing structured, dependent strategies. The `@given(strat, ...)` decorator binds one or more strategies to a test function’s arguments and drives the adaptive search engine. An example using `st.lists()` is shown in Figure 1.

A.3 Prompt templates

Below we present abridged versions of the two prompt templates used in the evaluation (full templates with Jinja2 conditionals are in `eval/prompts/` in the released repository). Per-problem variables (library name, version, scope files) are filled at runtime.

Baseline prompt

```
You have been given a Python library to test. The library implementation
may have one or more issues that the existing test suite does not detect.

## Your task

Write a test that fails when run against the current (potentially buggy)
```

```

version of the library, and would **pass** after the issue is fixed.
Save your final test to '/workspace/pbt_test.py'.

## Workspace layout

/workspace/
  lib/           -- the library under test (may contain a bug)
  docs/          -- API documentation and usage guide
  existing_tests/ -- existing test suite (all currently pass; do NOT modify)
  pytest.ini     -- pre-configured: lib/ is already on the Python path
  pbt_test.py    -- write your test here

## Instructions

Phase 1 -- Understand the library
1.0 Inspect the scope files listed under Scope.
1.1 Read the documentation in /workspace/docs/.
1.2 Study the existing tests to understand what is already covered.
1.3 Identify the public API: what functions/methods exist, what are their contracts?

Phase 2 -- Find uncovered behavior
2.1 Think about what behaviors the existing tests do NOT cover.
2.2 Consider boundary conditions, edge cases, and properties that should always hold.
2.3 Look for semantic invariants: properties that should be true for all valid inputs
    .

Phase 3 -- Write a failing test
3.1 Write a test using pytest that exercises uncovered behavior.
3.2 The test should fail because of the bug in the current library.
3.3 Run: timeout 60 python -m pytest /workspace/pbt_test.py --tb=short -v
3.4 If the test passes, rethink your approach and return to Phase 2.

Phase 4 -- Finalize
4.1 Make sure the test file is saved at /workspace/pbt_test.py.
4.2 Confirm the test fails with a meaningful assertion error.
4.3 Do NOT modify any files in /workspace/lib/ or /workspace/existing_tests/.

Notes:
- Issues are subtle: the library mostly works, but fails silently on certain inputs.
- There may be more than one issue. Try to cover as many as possible.
- Write focused test functions: one specific potential issue per function.

```

PBT (Hypothesis-scaffolded) prompt

```

You have been given a Python library to test using Property-Based Testing (PBT)
with the Hypothesis framework. The library implementation may have one or more
issues that the existing test suite does not detect.

Your goal is NOT to find the bug first and then write a test for it. Instead,
write the most comprehensive set of property-based tests you can derive purely
from the documentation -- covering every behavior, invariant, and edge case
described. If the implementation deviates from its specification anywhere,
Hypothesis will find it.

## Strategy

Step 1 -- Read the documentation thoroughly
Read all files in /workspace/docs/. For each function or class in scope, extract:
- The precise semantics: what does it do, what does it return?
- All stated invariants: what should always be true?
- Edge cases mentioned explicitly
- Relationships between functions: roundtrip properties, monotonicity
- Any numeric thresholds, size limits, or type-specific behavior

Do NOT read the source code at this stage. The documentation is your oracle.

Step 2 -- Write as many property tests as possible
For every documented behavior or invariant, write a @given test. Aim for 5+
distinct test functions. Prefer:
- Roundtrip properties: decode(encode(x)) == x
- Model-based oracles: compare to a simple Python reference
- Invariant preservation: after any operations, a structural invariant holds
- Commutativity / associativity / idempotence
- Monotonicity: larger input -> larger (or smaller) output
- Boundary behavior: empty collections, single elements, min/max values

Use @settings(max_examples=200, deadline=None) on every test.

Step 3 -- Cover edge cases explicitly

```

```

Beyond general properties, write targeted tests for documented edge cases.

Step 4 -- Run and save
Run: python -m pytest /workspace/pbt_test.py --tb=short -v
Tests that fail immediately indicate you found a bug -- excellent.
Tests that pass are still valuable. Save all tests regardless.

## Rules
- Do NOT modify /workspace/lib/ or /workspace/existing_tests/.
- Every test must use @given from Hypothesis (not just fixed inputs).
- Write multiple focused functions -- one property per function.
- The documentation is your primary oracle.
- Do not spend time hunting for a specific bug. Write tests that cover
  the spec broadly; Hypothesis will find the deviations.

```

A.4 Library list

The 40 canonical libraries span seven thematic domains: *serialization* (msgpack, cbor2, construct, yaml, tomlkit, lark, pycparser, pypdf); *data structures* (sortedcontainers, bidict, bintrees, cachetools, diskcache, intervaltree, multiset, boltons, portion, more-itertools, toolz); *date-time* (arrow, dateutil, babel, icalendar, pendulum); *type systems and schemas* (attrs, jsonschema, cattrs, marshmallow, pyrsistent); *numerics* (mpmath, pint, galois, sympy, networkx); *state machines* (transitions); and *parsing/HTML* (pyparsing, parso, html5lib, openpyxl, pyasn1). Version pins and per-library problem counts are in `paper/analysis/bug_metadata.csv`.

A.5 Bug-class to real-world analogue mapping

Table 2 groups the 365 injected bugs into eleven high-level classes derived from per-bug tags entries in `problem.yaml`. Each class corresponds to a bug pattern commonly observed in open-source Python projects.

Table 2: Injected-bug taxonomy across the 100 canonical problems. Columns are per-bug difficulty L1–L3.

Class	L1	L2	L3	Total
Off-by-one / boundary	23	30	12	65 (18%)
Wrong reference / expression	12	25	19	56 (15%)
Condition / logic inversion	12	30	7	49 (13%)
Wrong operator	13	28	3	44 (12%)
Argument / field order swap	6	19	17	42 (12%)
Missing operation	2	19	8	29 (8%)
Traversal / ordering	4	10	5	19 (5%)
Sign / direction error	4	6	6	16 (4%)
Protocol / domain-specific	0	6	9	15 (4%)
Wrong constant / value	7	5	2	14 (4%)
State corruption / mutation	2	2	4	8 (2%)
Other	2	4	2	8 (2%)
Total	87	184	94	365

A.6 Complete per-cell results heatmap

Figure 3 shows per-problem recall across all 100 problems (rows, sorted hardest at top) and all sixteen (model, mode) cells (columns, sorted weakest at left). Each cell color is the mean recall across the three runs for that cell, using the merged dataset (iter2 with clean-rerun substitutions where available). The ten hardest problems by mean cross-cell recall (BIDC-004 → TMLK-001, with BIDC-004 scoring 22.4%) dominate the ensemble-upper-bound ceiling in Section 5.2. The full underlying CSV is released at `paper/analysis/hardest_problems.csv`.

A.7 The .orig leak: detection and rerun policy

The initial harness (patch -p1) occasionally left *.orig backup files readable by the agent. We fixed this in commit 1048fd6 by unconditionally deleting backups after patch application.

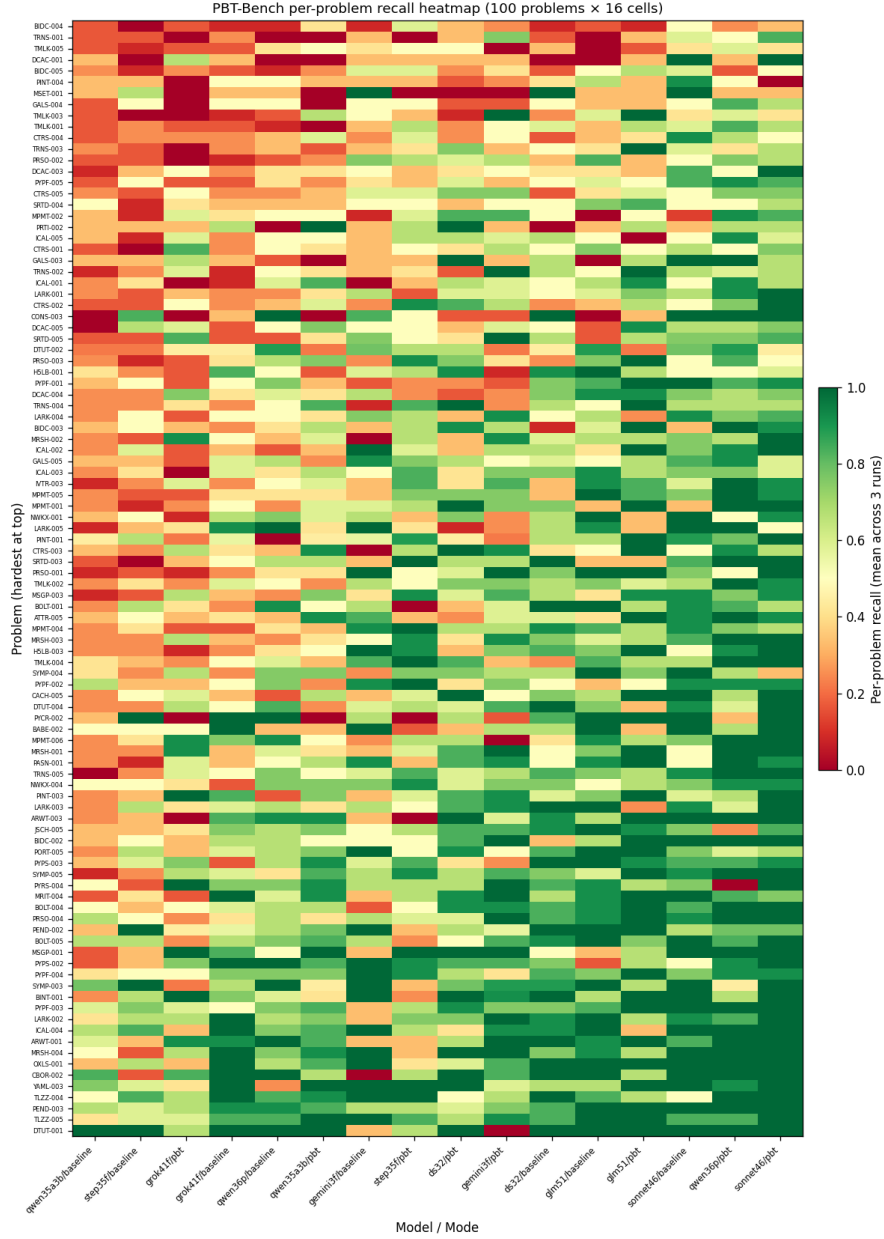


Figure 3: Per-problem recall, 100 problems × 16 cells. Problems sorted by mean cross-cell recall (hardest at top). Cells sorted by overall recall (weakest at left). Color is per-problem recall averaged across 3 runs.

Detection. We flag a workspace as “exploited” if its `chat.md` contains any regex match against five read-action patterns targeting `.orig` files (shell diff, `cat/head/tail`, `grep`, Python open, or `file_editor view`). In total, 383 of 4,800 workspaces (8.0%) matched.

Rerun policy. If *any* of the three runs for a (model, mode, problem) tuple contained exploitation, we reran *all three* runs on the post-fix harness. This affected 224 tuples (672 workspaces) across six models. The merged dataset used for all reported numbers contains zero residual pre-fix rows for any contaminated tuple.

Statistical effect. Within-problem paired tests (Wilcoxon signed-rank $p = 0.31$, $n = 160$ mixed tuples) show no detectable causal effect of exploitation on recall. The rerun is a hygiene measure, not a correction for inflated scores.

A.8 Failure-mode taxonomy: method and examples

Classifier. Failures were classified by a deterministic rule cascade implemented in `paper/analysis/build_failure_taxonomy.py`. The classifier consumes three signals per failed trial: (i) the submitted `pbt_test.py` source, examined for `@given`, `assume(...)` calls, default-strategy constructors like `st.integers()` or `st.text()`, and weak-assertion patterns (`is not None`, `len(x) >= 0`, `isinstance`, `round-trip encode/decode`); (ii) the per-function pass pattern on (buggy, fixed) library versions from `bugs_result.json`; (iii) `chat.md` for the complete agent interaction history. The cascade and its outputs across 320 sampled failures are in `paper/analysis/failure_taxonomy.csv`.

Sampling. For each of the 16 (model, mode) cells we sampled 20 failures uniformly over problems, subject to `test_file_found=True AND found=False`. Across cells: 320 samples.

Representative examples.

- **Assume Misuse** — Sonnet 4.6 PBT on H5LB-003 bug 2: `assume(val1 != val2)` filters out the exact duplicate-value case that is the bug’s trigger.
- **Assume Misuse** — Sonnet 4.6 PBT on SRTD-005 bug 2: `assume(query not in sl)` — the bug is specifically about duplicate-query lookups.
- **Wrong Strategy Range** — DeepSeek V3.2 PBT on DCAC-005: `@given(st.integers())` for a cache-expire timestamp, making triggers at boundary seconds astronomically unlikely.
- **Wrong Strategy Range** — DeepSeek V3.2 PBT on CTRS-001: `st.text(alphabet=st.characters(whitelist_categories=('L','N','P','S','Z')))` excludes control characters that the bug targets.
- **Flaky Oracle** — Grok 4.1 Fast Baseline on ICAL-003: oracle reads the serialized string back via `Calendar.from_ical(ical_str.encode())` — the same buggy parser that produced the value.
- **Incorrect Assertion** — DeepSeek V3.2 Baseline on H5LB-003: a test asserts that `A`; parses to `'A'`, but the test’s expected value is itself what the buggy `consumeNumberEntity` produces — fails on both buggy and fixed libraries.
- **Setup Error** — DeepSeek V3.2 PBT on SYMP-005: duplicate `from hypothesis import strategies as st` and `import hypothesis.strategies as st` produce a collection-time crash; test functions never run.
- **Under-Specified Property** — DeepSeek V3.2 Baseline on PYCR-002: `assert return_match is not None` checks only the existence of some return statement, silent on whether it has the right shape.

Model-specific observations.

- **Sonnet 4.6** and **GLM 5.1** over-use `assume()`: they understand Hypothesis well enough to write generators, but also willingly filter out the trigger region.
- **Qwen 3.5-30B-A3B** and **Gemini 3 Flash** mostly fail on unbounded default strategies (45–50% Wrong Strategy Range).
- **Step 3.5 Flash Baseline** skews Overly Concrete (80%) — it writes a handful of hardcoded examples when it could write a property test.

Taxonomy adequacy. After two rule-cascade refinements (splitting baseline “overly concrete” failures by their (buggy, fixed) pass pattern; excluding from hypothesis import assume false-positives from the Assume-Misuse regex), zero failures landed in the OTHER bucket. We take this as evidence that the eight categories are collectively sufficient for PBT failure analysis at the granularity of a deterministic classifier.

A.9 L1–L3 difficulty validation (empirical)

Figure 4 plots the confusion matrix of author-assigned L1–L3 against empirical difficulty buckets (Easy: Sonnet-Baseline passrate $\geq 2/3$; Medium: $\geq 1/3$; Hard: $< 1/3$). Mean Sonnet-Baseline passrate is 79% for L1 bugs ($n = 87$), 78% for L2 ($n = 184$), and 68% for L3 ($n = 94$). Medians are 1.0 for all three levels (Sonnet Baseline finds these on all three runs when it finds them at all), so the difficulty signal is carried by the mean, not the median; the distinction emerges in the left tail of each distribution.

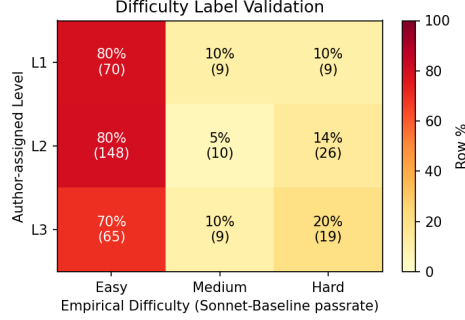


Figure 4: Author-assigned difficulty (rows) against empirical difficulty bucketed by Sonnet-Baseline passrate (columns). Each row sums to 100%. Cell labels are %±(count). L3 bugs concentrate in the Hard bucket; L1 bugs concentrate in Easy.

The L1/L2 passrate gap is tighter (79% vs. 78%) than the L2/L3 gap (78% vs. 68%). This is expected: L1 and L2 both involve input-constraint reasoning (differing in the number of simultaneous constraints), while L3 additionally requires cross-function protocol understanding — a qualitatively different skill. The gap widens when averaging across all models (L1: 66%, L2: 60%, L3: 51%), confirming that the three levels separate empirically distinct difficulty strata. Labels are assigned prospectively at design time, before any model evaluation, to avoid circularity.

Table 3 reports per-cell recall broken down by difficulty level. In 15 of 16 cells, recall decreases monotonically with difficulty (L1 > L2 > L3); the sole exception is GLM 5.1 Baseline, where L2 exceeds L1 by 2.2 pp — within the bootstrap CI width.

Per-difficulty Δ patterns. The following observations are exploratory: per-difficulty Δ values are not accompanied by per-cell confidence intervals, so small differences should be interpreted cautiously.

(a) Diminishing-return pattern. Qwen 3.5-30B-A3B shows the clearest gradient: +35.2/+25.4/+14.8 pp on L1/L2/L3. One interpretation is that the scaffold supplies a framework choice and input-constraint template sufficient for L1 bugs (single-constraint boundary checks) but insufficient for L3 bugs requiring cross-function protocol reasoning. This pattern is specific to Qwen 3.5-30B-A3B; Qwen 3.6 Plus (+25.3/+25.5/+22.7 pp) is near-flat and grouped with the uniform-benefit pattern below.

(b) Uniform-benefit pattern. Step 3.5 Flash and Qwen 3.6 Plus achieve near-constant Δ across all three difficulty levels ($\approx +22.5$ and $\approx +24.8$ pp respectively). This is consistent with the scaffold supplying a coverage discipline (“write five focused tests, one invariant per function”) that benefits these models roughly equally regardless of bug difficulty.

(c) Difficulty-amplified degradation. DeepSeek V3.2 and Grok 4.1 Fast show negative overall Δ , with greater degradation at higher difficulty (DeepSeek: +2.7/−4.0/−7.8 pp; Grok: −5.7/−5.2/−10.6 pp). One possible explanation is strategy over-specification: the PBT template may constrain the input search in ways that are tolerable for L1 bugs but exclude the narrower trigger regions of L2–L3 bugs, consistent with the elevated search-related failure rates documented for these models in Appendix A.12. Gemini 3 Flash shows a weaker version at L3 only (−0.4 pp, likely within noise) while retaining positive Δ at L1/L2 (+8.8/+12.9 pp), making it a borderline case between patterns (b) and (c).

Table 3: Bug recall (%) by difficulty level for all 16 (model, mode) cells, sorted by PBT overall recall (descending). Δ = PBT – Baseline in pp; negative values indicate PBT hurts. n : L1 = 87, L2 = 184, L3 = 94 bugs. *Note*: recall here is bugs-found / bugs-total pooled across all bugs in each difficulty stratum (bug-level average), whereas Table 1 reports problem-level average recall (each problem weighted equally); the two metrics differ slightly when problems contain unequal numbers of bugs.

Model	Mode	L1	L2	L3	Overall
Sonnet 4.6	PBT	93.1	83.2	74.5	83.3
	Baseline	79.3	77.5	68.1	75.5
	Δ	+13.8	+5.7	+6.4	+7.8
Qwen 3.6 Plus	PBT	87.0	79.3	67.7	78.2
	Baseline	61.7	53.8	45.0	53.4
	Δ	+25.3	+25.5	+22.7	+24.8
GLM 5.1	PBT	75.9	74.5	68.4	73.2
	Baseline	68.6	70.8*	62.1	68.0
	Δ	+7.3	+3.7	+6.3	+5.2
Gemini 3 Flash	PBT	70.9	67.8	53.9	64.9
	Baseline	62.1	54.9	54.3	56.4
	Δ	+8.8	+12.9	−0.4	+8.5
DeepSeek V3.2	PBT	70.5	61.8	49.6	60.7
	Baseline	67.8	65.8	57.4	64.1
	Δ	+2.7	−4.0	−7.8	−3.4
Step 3.5 Flash	PBT	65.9	59.4	52.1	59.1
	Baseline	43.3	37.1	29.4	36.6
	Δ	+22.6	+22.3	+22.7	+22.5
Qwen 3.5-30B-A3B	PBT	69.3	56.2	41.8	55.6
	Baseline	34.1	30.8	27.0	30.6
	Δ	+35.2	+25.4	+14.8	+25.0
Grok 4.1 Fast	PBT	52.5	43.7	29.8	42.2
	Baseline	58.2	48.9	40.4	48.9
	Δ	−5.7	−5.2	−10.6	−6.7

*Sole non-monotone cell (L2 > L1 by 2.2 pp; within bootstrap CI).

(d) Large- Δ -at-L1 pattern. Sonnet 4.6 shows its largest absolute Δ on L1 (+13.8 pp), smaller at L2/L3 (+5.7/+6.4 pp). Sonnet 4.6 Baseline already achieves 79.3% on L1 — leaving limited headroom — so the large L1 gain is notable; diminishing gains at L2/L3 may partly reflect that ceiling. GLM 5.1 shows the same ordering (+7.3/+3.7/+6.3 pp) but its Baseline L1 of 68.6% is not near ceiling, so the explanation is less clear-cut.

We speculate that these patterns are consistent with two roles for PBT scaffolding: a *framework bootstrapping* role (consistent with patterns b and d) that supplies the Hypothesis API and coverage discipline; and a *constraint elicitation* role (consistent with pattern a) that substitutes for input-reasoning skills with diminishing returns as bug complexity grows. Whether these roles are genuinely separable, and why some models are harmed by scaffolding, would require controlled ablations — for instance, prompts that supply only the framework name versus prompts that add the full property taxonomy.

A.10 Cross-model bug-finding complementarity

Not every hard bug is hard for every model. The three galois library problems (GALS-003-005) illustrate this: each requires deep algebraic domain knowledge (finite-field arithmetic, LFSR protocols, polynomial factorization), yet model rankings are inconsistent across them.

- **GALS-003** — Sonnet Baseline achieves 100% recall while several weaker models score $\leq 33\%$; yet DeepSeek PBT and Qwen 3.6 PBT also reach 100%.
- **GALS-004** — Qwen 3.6 PBT leads at 83%, while Sonnet PBT scores 67% and DeepSeek PBT only 17%. No model achieves full recall.

- **GALS-005** — Gemini Baseline (92%) and Qwen 3.6 PBT (92%) lead; Sonnet Baseline scores 83% but Sonnet PBT drops to 58%.

This is not a general-capability inversion: Sonnet dominates overall recall. Rather, it indicates that PBT-Bench exposes problems for which specific model architectures or scaffolded reasoning paths may be differentially advantaged. For benchmark users, the actionable consequence is that *combining test files across models* is a recall-enhancing strategy.

A.11 Never-found bugs

Two of 365 bugs are never reliably found (found on ≥ 2 of 3 runs) by any of the 16 (model, mode) cells:

- **BIDC-004 bug_3** — a stateful bidict invariant violation requiring a specific sequence of put/forceput operations that triggers an internal node-count desync. The trigger probability under default strategies is near zero.
- **TMLK-005 bug_1** — a tomlkit container indexing bug where `unwrap()` silently drops a nested inline table when a specific combination of dotted keys and inline-table nesting is present. Requires a composite strategy generating nested TOML structures of depth ≥ 3 .

Both are L3 bugs requiring cross-operation protocol knowledge. Their resistance to all evaluated models confirms the benchmark ceiling is meaningful.

A.12 Failure-mode taxonomy (full)

To characterize *how* agents fail, we sampled 20 failed trajectories per (model, mode) cell (320 samples, uniform over problems) and classified each into one of eight mutually-exclusive failure modes. Classification is rule-based over the submitted `pbt_test.py` combined with per-function buggy/fixed pass patterns; a PBT-specific refinement of the MAST taxonomy of Pan et al. [2025] and the trajectory-level analysis of Merrill et al. [2026]. Full categorization rules and per-sample outputs are in Appendix A.8 and the released `paper/analysis/failure_taxonomy.csv`.

Baseline vs PBT mode shows categorically different failure profiles. In Baseline mode ($n = 160$ classified failures), 59% of failures are **Incorrect Assertion** (the test’s expected values are wrong — assertions fail against *both* buggy and fixed libraries because the agent fabricated incorrect outputs), 32% **Overly Concrete Test** (a few hardcoded inputs that happen not to intersect the bug’s trigger region), 8% **Flaky Oracle** (agent re-uses the library under test as its own oracle, typically via a round-trip encode/decode whose asymmetric failure is masked), and 1% **Under-Specified Property**. This pattern says Baseline agents *correctly construct tests* but aim them at the wrong invariant.

In PBT mode ($n = 160$), failure shifts to *search pathologies*. The single largest PBT-failure category is **Assume Misuse** (31% across PBT cells, peaking at 75% for GLM 5.1, 50% for DeepSeek V3.2, and 40% for Sonnet 4.6): the agent successfully designs a `@given` test but calls `assume()` on a predicate that happens to *exclude the bug’s trigger region* — e.g. `assume(val1 != val2)` in a test for a duplicate-value bug. The Hypothesis engine then never samples the triggering input. A second major PBT failure is **Wrong Strategy Range** (31% across PBT cells, peaking at 45% for Qwen 3.5-30B-A3B and 50% for Gemini 3 Flash): `st.integers()` where `st.integers(min_value=32768, max_value=65535)` is needed, for example. In PBT mode, Baseline’s dominant Incorrect Assertion drops to 21% (from 59%) and Overly Concrete Test vanishes (0%), because the prompt’s `@given` template makes concrete-only tests syntactically unusual.

This is a previously-unreported diagnostic specific to property-based-testing evaluation: even when an agent correctly chooses Hypothesis, its most common failure mode is filtering out the counterexample region with `assume()`. Table 4 reports per-cell percentages; concrete examples are in Appendix A.8.

Implication for prompt design. The **Assume Misuse** finding is actionable: a PBT-scaffolding prompt that warns explicitly “do not use `assume()` to skip inputs that look suspicious; inputs that look suspicious are exactly the ones that expose the bug” is likely to recover a substantial fraction of PBT-mode failures across most tested models. We leave empirical verification to future work.

Table 4: Failure-mode distribution per (model, mode) cell, as percentages of 20 sampled failures per cell. WS = Wrong Strategy Range, US = Under-Specified Property, AM = Assume Misuse, OC = Overly Concrete Test, FO = Flaky Oracle, SE = Setup Error, IA = Incorrect Assertion, Ot = Other. Bold = row’s largest category.

Model	Mode	WS	US	AM	OC	FO	SE	IA	Ot
Sonnet 4.6	Baseline	0	0	0	20	10	0	70	0
Sonnet 4.6	PBT	25	0	40	0	20	0	15	0
GLM 5.1	Baseline	0	0	0	30	15	0	55	0
GLM 5.1	PBT	25	0	75	0	0	0	0	0
Qwen 3.6 Plus	Baseline	0	0	0	25	0	0	75	0
Qwen 3.6 Plus	PBT	20	0	40	0	20	0	20	0
DeepSeek V3.2	Baseline	0	5	0	45	10	0	40	0
DeepSeek V3.2	PBT	20	0	50	0	0	5	25	0
Gemini 3 Flash	Baseline	0	0	0	10	5	0	85	0
Gemini 3 Flash	PBT	50	5	0	0	20	0	25	0
Step 3.5 Flash	Baseline	0	5	0	80	5	0	10	0
Step 3.5 Flash	PBT	40	0	40	0	10	0	10	0
Qwen 3.5-30B-A3B	Baseline	0	0	0	45	5	0	50	0
Qwen 3.5-30B-A3B	PBT	45	0	0	0	15	5	35	0
Grok 4.1 Fast	Baseline	0	0	0	0	15	0	85	0
Grok 4.1 Fast	PBT	20	0	5	0	25	15	35	0

A.13 Run-to-run stability (full)

All headline metrics are computed over 3 independent runs per (model, mode) cell and reported with 95% percentile bootstrap confidence intervals (1,000 resamples over problems $\times$ runs). Across the 16 cells in Table 1, the Recall CI half-width ranges from 2.7 pp (Sonnet Baseline) to 4.9 pp (Step PBT), with a median of 3.8 pp. These intervals are narrow enough that every sign claim in Section 5 — including all PBT-vs-Baseline Δ values — survives resampling.

Residual variance is traceable to Hypothesis’s internal search procedure, not agent non-determinism. The GALS-003 case discussed in Appendix A.10 is the canonical example: the agent’s strategy correctly targets polynomials in $\text{GF}(q)$, but only some random seeds generate one with the specific ≥ 3 -distinct-degree-factor structure required to trigger the bug within the default `max_examples=200` budget. For problems with known narrow triggers, raising `max_examples` to 1,000 or higher would tighten the intervals further; a controlled study of this budget parameter is left to future work.

Post-hoc prompt check. After the main runs, we wrote two paraphrased variants of each prompt (`baseline_v{2,3}.j2` and `pbt_hypothesis_v{2,3}.j2`) and ran Sonnet 4.6 on all 100 problems under each variant (single run per variant) to check that headline recall is not an artifact of a particular phrasing.

Baseline mode. Original (3-run mean): 0.765; v2: 0.787; v3: 0.650. The original and v2 are within seed noise of each other (the three seeds of the original prompt span 4.3 pp), while v3 is clearly worse.

PBT mode. Original (3-run mean): 0.834; v2: 0.762; v3: 0.842. The original and v3 are within seed noise (the three seeds of the original span 7.6 pp), while v2 is clearly worse.

In both modes we kept the original prompt for the reported numbers. We flag that Baseline v2 and PBT v3 are equally defensible choices and release all candidates in `eval/prompts/`; a full sensitivity study (more models, more seeds per variant) is left to future work. To our knowledge, no comparable benchmark paper (SWE-bench, SWT-Bench, Terminal-Bench, LiveCodeBench) reports a multi-model prompt-paraphrase sweep; our single-model check already exceeds field norms. The large v3 Baseline drop (−11.5 pp) is most plausibly attributable to an inferior paraphrase rather than fundamental instability, given that the concurrent v2 variant deviates by only +2.2 pp.

A.14 Eval-phase timeout

One tuple (`glm51` $\times$ `PBT` $\times$ `run 3` $\times$ `GALS-005`) hit the harness’s eval-phase wall-clock limit. The agent produced 96 @given test functions; during the per-function $F \rightarrow P$ scoring loop, repeated pytest 300-second timeouts accumulated past the 1,800-second eval-phase budget. That trajectory’s four bugs are recorded as `detection_method=eval_phase_timeout / found=False` rather than re-tried indefinitely. This is the sole abort in the 4,800-trajectory corpus. The harness now exposes an `-eval-timeout` flag (default 1,800 s) to make this failure mode explicit for future runs.

A.15 Compute resources and cost

All evaluations ran on a single 96-core, 512 GB RAM CPU node (no GPU) with up to 20 concurrent Apptainer containers. Total wall-clock cost was approximately 200 node-hours. Aggregate API cost across the eight models was \$2,288, broken down as follows: Sonnet 4.6 PBT \$631; GLM 5.1 PBT \$364; Sonnet 4.6 Baseline \$308; all other cells <\$250 each.